



Trabalho Prático  
Meta 2  
**Sistemas Operativos II**

Renato Alexandre Oliveira Craveiro - 2018011392

Jorge Daniel Costa Soares - 2021140196

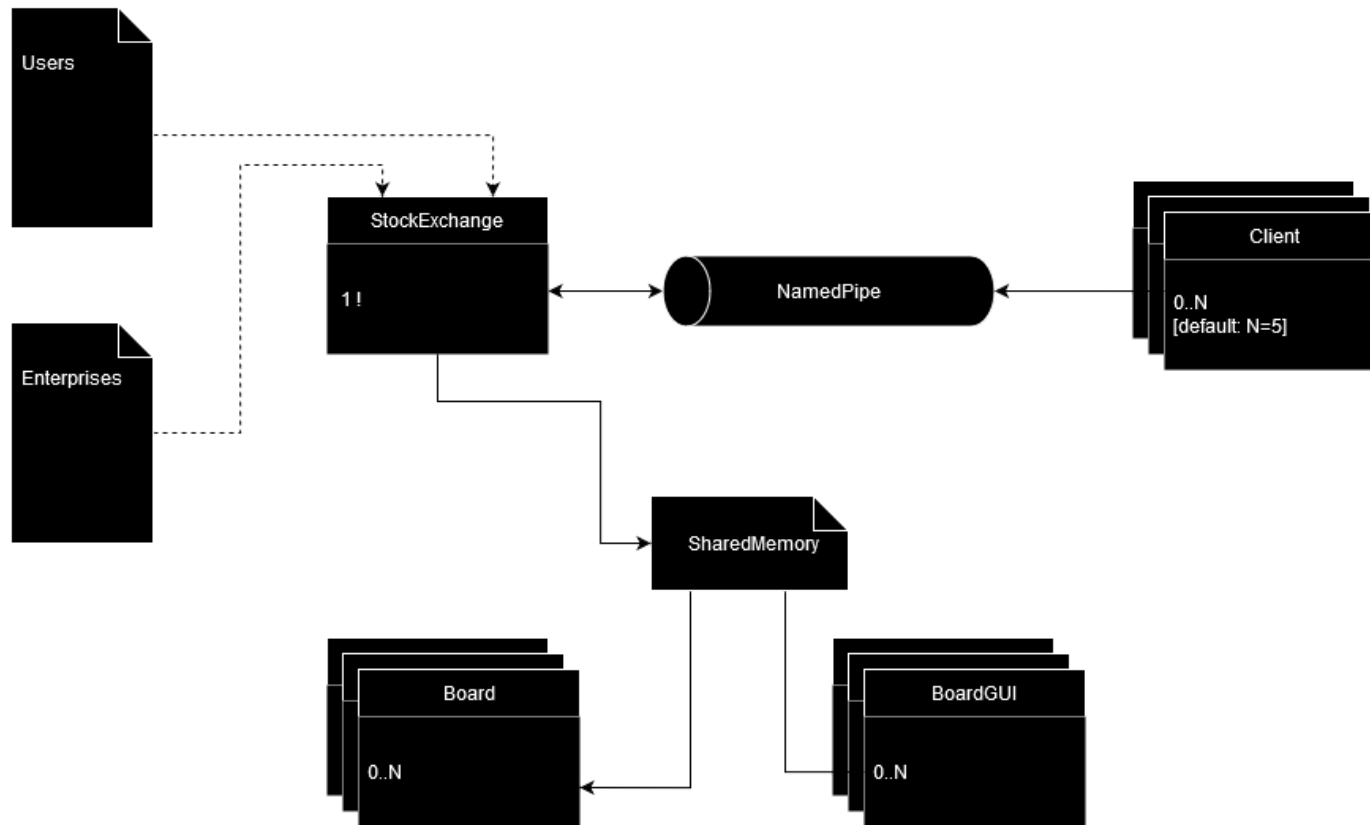
Lic. Eng. Informática | ISEC | Ano-Letivo 2023/2024

---

# Índice

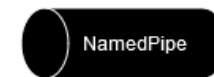
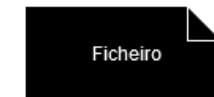
|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Arquitetura do Sistema e Mecanismos de Comunicação.....</b> | <b>2</b>  |
| <b>Síntese Lógica de Funcionamento - Flowchart.....</b>        | <b>3</b>  |
| <b>Client Lifecycle - Flowchart.....</b>                       | <b>4</b>  |
| <b>Board Lifecycle - Flowchart.....</b>                        | <b>5</b>  |
| <b>Server Lifecycle - Flowchart.....</b>                       | <b>6</b>  |
| <b>Ficheiros de dados.....</b>                                 | <b>7</b>  |
| <b>Estruturas de dados.....</b>                                | <b>7</b>  |
| Variáveis Globais.....                                         | 7         |
| Estruturas.....                                                | 7         |
| <b>Requisitos Implementados.....</b>                           | <b>10</b> |
| <b>Manual de utilização.....</b>                               | <b>11</b> |
| Requisitos.....                                                | 11        |
| Inicialização.....                                             | 11        |
| Server.....                                                    | 11        |
| Comandos Disponíveis.....                                      | 11        |
| Cliente.....                                                   | 12        |
| Comandos Disponíveis.....                                      | 12        |
| Board.....                                                     | 13        |
| <b>Mecanismos de Sincronização.....</b>                        | <b>14</b> |
| <b>Bibliografia.....</b>                                       | <b>15</b> |

# Arquitetura do Sistema e Mecanismos de Comunicação



## Legenda

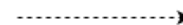
**Processo**  
 x.y [!]  
 x: mínimo processos que podem estar a correr  
 y: máximo de processos que podem correr  
 !: obrigatório para qualquer processo envolvido correr



Comunicação/transmissão de dados para ficheiros "fictícios" (Shared Memory, Named Pipes, ...)



Comunicação/transmissão de dados para ficheiros "físicos" (ficheiros reais - .txt, .data...)



# Síntese Lógica de Funcionamento - Flowchart

## 1. Inicialização do Programa:

- O programa é iniciado pela linha de comando, onde o nome do arquivo de utilizadores é fornecido.

## 2. Leitura de Utilizadores:

- O arquivo de utilizadores é lido, contendo informações como username, password e saldo. Limitado a 20 utilizadores, sem usernames repetidos.

## 3. Leitura de Empresas:

- O administrador do Server indica empresas, número de ações iniciais e preço inicial através da consola ou de um arquivo de texto, sem nomes de empresas repetidos, limitado a 30 empresas.

## 4. Gestão de Carteira de Ações:

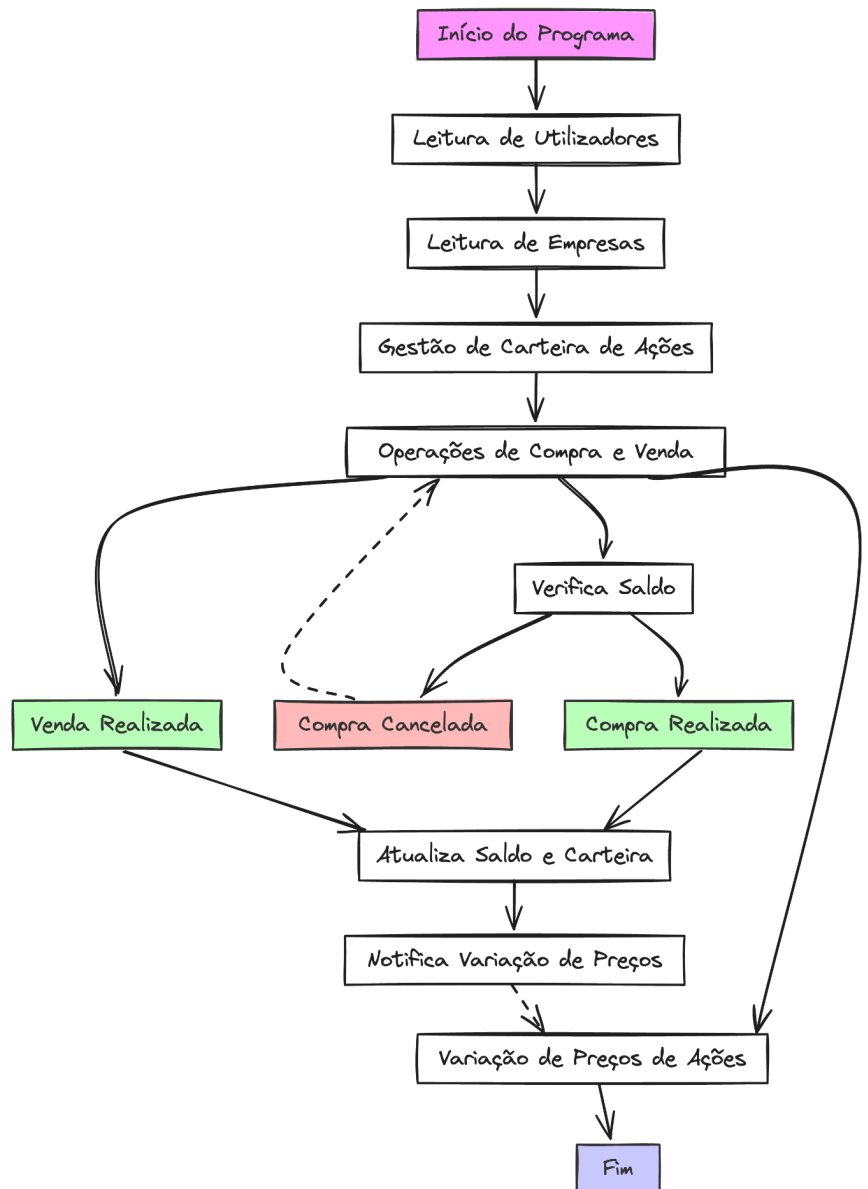
- Cada utilizador pode ter ações de até 5 empresas diferentes. Inicialmente, não possuem ações.

## 5. Operações de Compra e Venda:

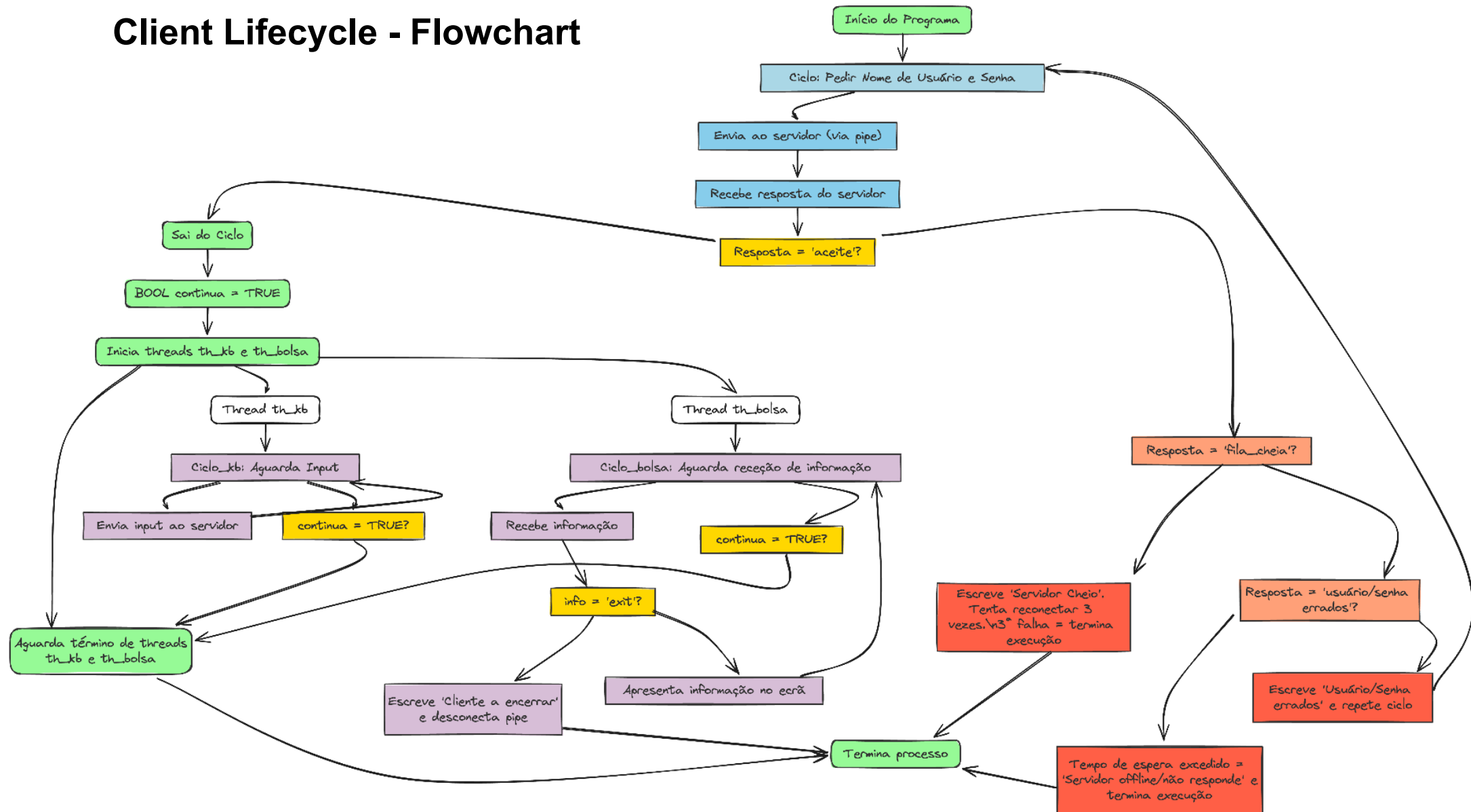
- Utilizadores podem solicitar compras e vendas através de um programa cliente.  
- A compra requer saldo suficiente e pode ser satisfeita por ações disponíveis ou ofertas de venda pendentes. Falta de saldo cancela a operação.  
- Vendas revertem valor ao saldo do vendedor.

## 6. Variação de Preços de Ações:

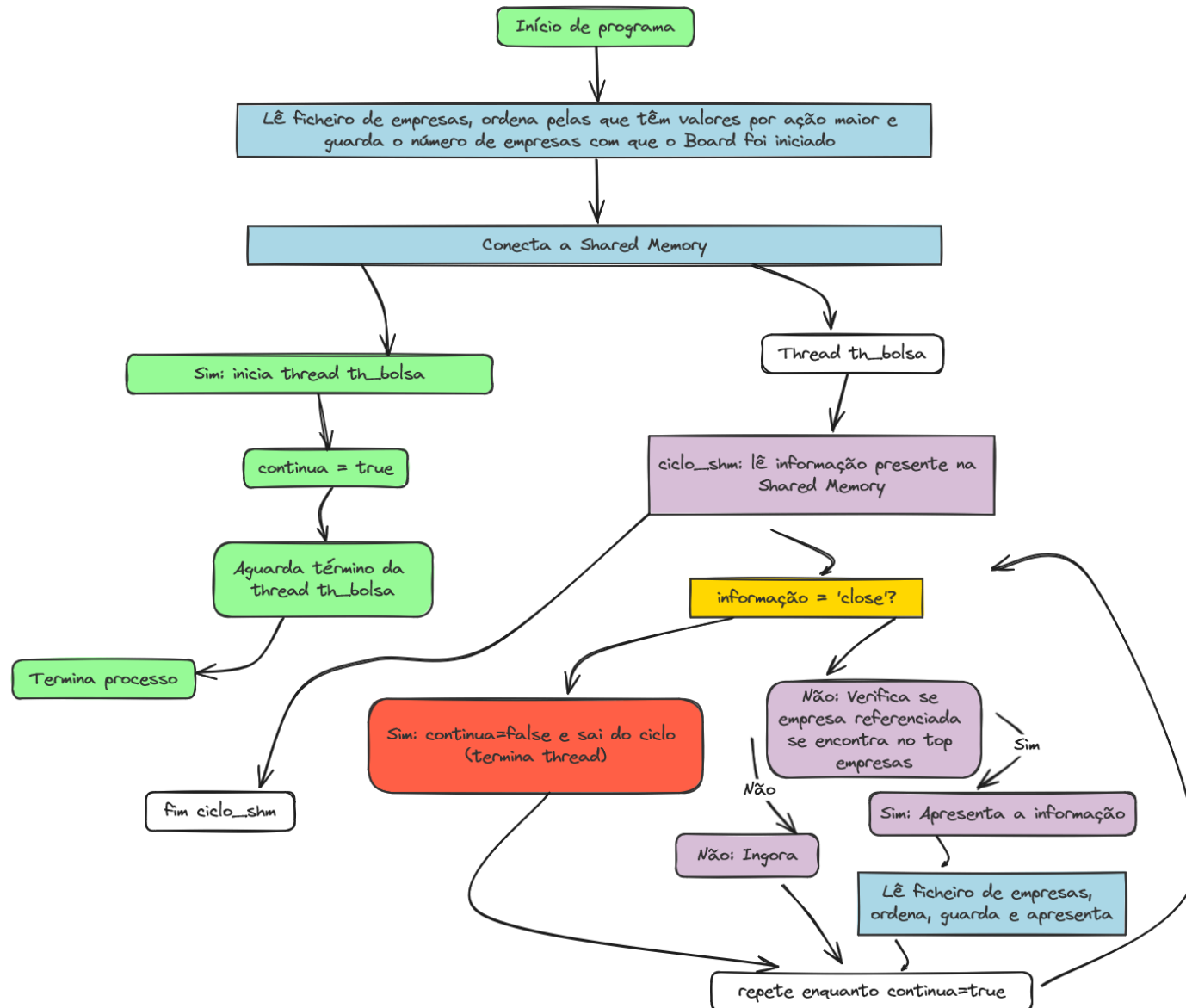
- Compras aumentam e vendas diminuem o preço das ações.
- Alterações podem ser naturais, aleatórias ou administrativas.
- Mudanças são comunicadas aos clientes.



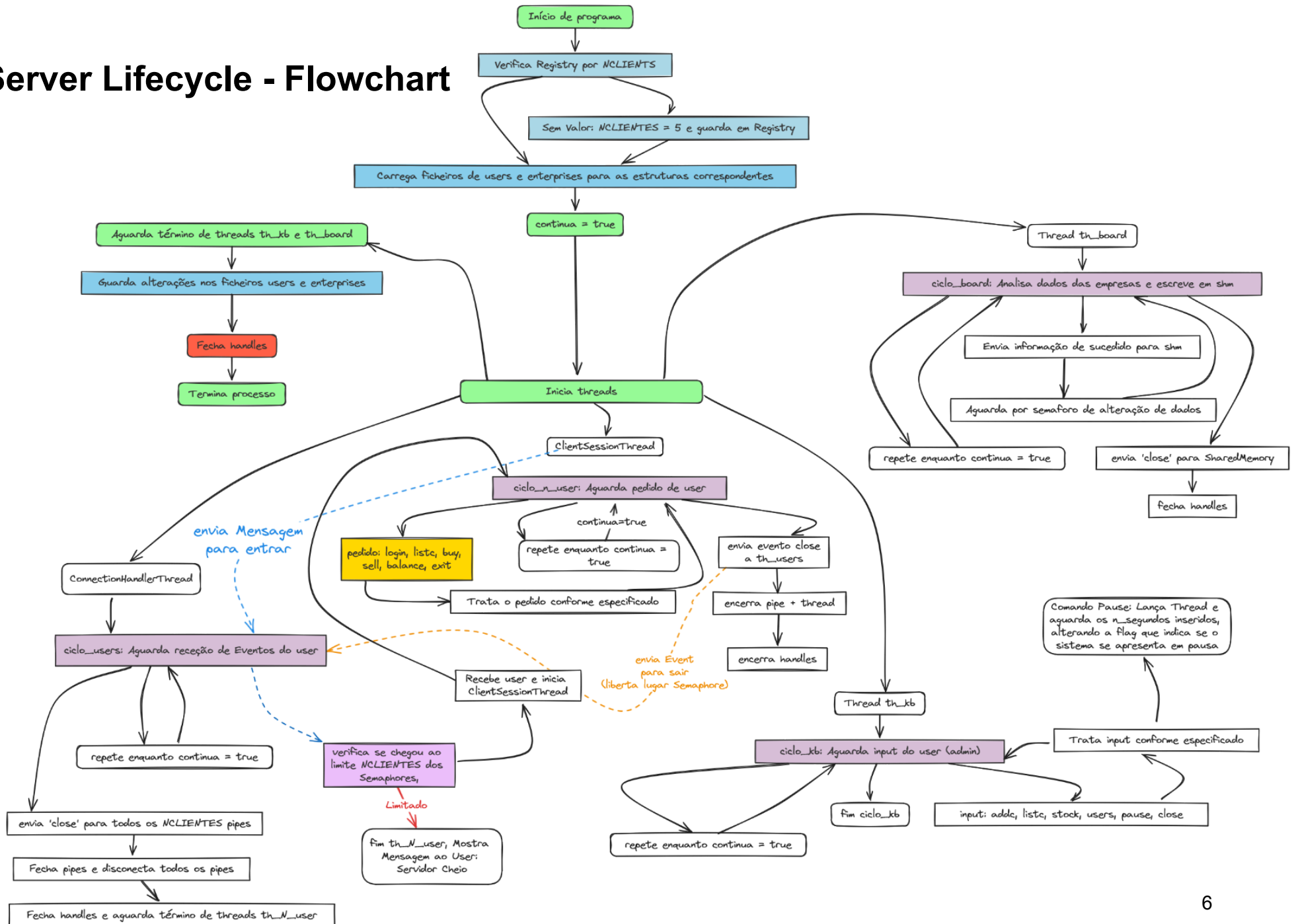
# Client Lifecycle - Flowchart



# Board Lifecycle - Flowchart



# Server Lifecycle - Flowchart



# Ficheiros de dados

*Ficheiros carregados no início da execução e guardados no fim*

## users.txt

name password balance\n

## companies.txt

entp\_name n\_stock stock\_value\n

# Estruturas de dados

## Variáveis Globais

```
#define SHM_NAME TEXT("BoardSHM")
#define MUTEX_NAME TEXT("RWMutex")
#define EVENT_NAME TEXT("NEWBOARDMSG")
#define USERSFILE TEXT("users.txt")
#define COMPANIESFILE TEXT("companies.txt")
#define REGISTRY_NCLIENTES_KEY TEXT("SO2")
#define REGISTRY_NCLIENTES_VALUE TEXT("NCLIENTES")
#define MY_STOCK_PIPE TEXT("\\\\.\\pipe\\MyStockPipe")
#define BUFFER_SIZE 512 // Adjust size as needed
#define MAX_TCHAR 256
#define MAX_USERS 50
#define MAX_COMPANIES 10
#define MAX_COMPANY_PER_USER 5
#define MAX_USERS_ONLINE 2
#define MSGTEXT_SZ 1024
#define LOGIN_SUCCESS TEXT("LOGIN_SUCCESS")
#define LOGIN_FAIL TEXT("LOGIN_FAIL")
#define LOGIN_WAIT TEXT("LOGIN_WAIT")
#define LOGIN_ALLOWED TEXT("LOGIN_ALLOWED")
#define _SECOND 1000000
#define MSFBUFSIZE sizeof(SharedMsg)
```

## Estruturas

```
// utils.h
typedef struct _MSG {
    TCHAR szMessage[MSGTEXT_SZ];
    TCHAR userName[MSGTEXT_SZ];
}SharedMsg;
```

```

typedef struct _ControlData {
    HANDLE hMapFile;
    SharedMsg* sharedMsg;
    int threadMustContinue;
    HANDLE hnewMsg;
    HANDLE hExitEvent;
    HANDLE hRWMutex;
}ControlData;

typedef struct _Company {
    TCHAR name[MAX_TCHAR];
    double value;
    DWORD nShares;
}Company;

typedef struct _User {
    TCHAR name[MAX_TCHAR];
    TCHAR password[MAX_TCHAR];
    double balance;
    HANDLE hPipe; // Handle for communication pipe for each user
    BOOL online;
    Company companies[MAX_COMPANY_PER_USER];
    int nCompanies;
}User;

// Server
typedef struct _GlobalData {
    ControlData* cdata; // Shared memory and synchronization objects
    Company* companies;
    User* users;
    int nCompanies;
    int nUsers;
    HANDLE hSemaphore; // Semaphore to limit number of concurrent users
    HANDLE hMutex; // Mutex for synchronization
    BOOL onPause; // Flag to pause stock transactions
    HANDLE hPipe; // Temporary Pipe handle for testing
    int nSecondsPause;
    int activeUsers; // Current number of active users
    HANDLE hPauseThread; // Handle to the pause thread
    HANDLE hShm;
    HANDLE hWaitableTimer; // Handle to the waitable timer
    HANDLE hMutexPipe; // Mutex to control access to the pipe
    TCHAR latestTransaction[MSGTEXT_SZ]; // Latest transaction data
    TCHAR* lastCompanyTrName; // Last company transaction name
} GlobalData;

```

```

typedef struct _UserWithGlobalData {
    User* user;
    GlobalData* gd;
} UserWithGlobalData;

// Board
typedef struct {
    ControlData* cdata;
    int numCompaniesOnDisplay;
    int actualCompaniesN;
    Company* companies;
} BoardData;

//Client
typedef struct {
    HANDLE hPipe;
    BOOL shouldRun;
    HANDLE hEvent; // Event to signal threads to exit
    HANDLE hResponseEvent; // Event to signal when a response is
received
} ThreadData;

```

# Requisitos Implementados

| ID | Descrição Funcionalidade/Requisito                                                 | Estado           |
|----|------------------------------------------------------------------------------------|------------------|
| 1  | Inicialização e carregamento dos dados de Clients a partir de arquivo              | Implementado     |
| 2  | Inicialização e carregamento dos dados das empresas a partir de arquivo            | Implementado     |
| 3  | Comunicação inter processos usando memória compartilhada para a troca de mensagens | Implementado     |
| 4  | Uso de mutex para garantir acesso exclusivo à memória compartilhada                | Implementado     |
| 5  | Criação e manipulação de eventos para sincronização das operações                  | Implementado     |
| 6  | Processamento e resposta a comandos do Client no programa Server                   | Implementado     |
| 7  | Implementação de thread para recebimento de mensagens no Server                    | Implementado     |
| 8  | Envio de mensagens do Server para o Board utilizando memória compartilhada         | Implementado     |
| 9  | Gerenciamento de estados e transações de compra e venda de ações                   | Implementado     |
| 10 | Autenticação do Client no sistema                                                  | Implementado     |
| 11 | Listagem de todas as empresas com detalhes de ações disponíveis e preço atual      | Implementado     |
| 12 | Realização de operações de compra e venda de ações                                 | Implementado     |
| 13 | Atualização dinâmica dos preços das ações baseada em oferta e demanda              | Implementado     |
| 14 | Listagem de todos os users e seus saldos                                           | Implementado     |
| 15 | Comandos para pausar e retomar operações de compra e venda                         | Implementado     |
| 16 | Funcionalidade de encerramento seguro dos sistemas                                 | Implementado     |
| 17 | Interface de linha de comando para todos os programas                              | Implementado     |
| 18 | Implementação da GUI para visualização dinâmica das ações                          | Não Implementado |
| 19 | Utilização do Registry para carregar o nível máximo de Clientes Ativos             | Implementado     |
| 20 | Valor de N Empresas é especificado através de um argumento da CLI                  | Implementado     |

# Manual de utilização

## Requisitos

- Ter acesso a um terminal de comando no sistema operativo Windows onde a aplicação está hospedada.
- Ter os ficheiros “users.txt” e “companies.txt” na diretoria da aplicação devidamente preenchidos
  - “users.txt”: em cada linha deverá ter a seguinte informação “username password saldo\n”
  - “companies.txt”: em cada linha deverá ter a seguinte informação “nome\_empresa num\_acoes valor\_por\_acao\n”

## Inicialização

De forma a executar cada uma das aplicações deverá abrir a Linha de comandos (cmd.exe) e colocar-se na localização onde se encontra a aplicação (*cd X:\caminho\para\aplicacao*).

Dentro da diretoria deverá executar o comando respetivo a cada aplicação:

- “.\Server.exe”
- “.\Clientes.exe”
- e “.\Board.exe [NUM\_EMPRESAS]”, onde NUM\_EMPRESAS representa a quantidade de empresas a ser apresentado na aplicação Board a executar

Poderá começar a utilizar comandos da aplicação a partir do momento em que for apresentado o texto “>” no terminal (exceto na aplicação Board que não permite utilização de comandos).

## Server

### Comandos Disponíveis

1. **Adicionar Empresa**
  - **Comando:** `addc <nome_empresa> <num_ações> <preço-ação>`
  - **Descrição:** Adiciona a empresa “nome\_empresa” com o número de ações e valor por ação indicados
  - **Exemplo:** `addc IBM 100 10.5`
2. **Listar Empresas**
  - **Comando:** `listc`
  - **Descrição:** Lista todas as empresas registradas na Server, mostrando o nome, o número de ações disponíveis e o preço atual das ações.
  - **Exemplo:** `listc`
3. **Redefinir custo das ações de uma empresa**
  - **Comando:** `stock <nome-empresa> <preço-ação>`
  - **Descrição:** Altera o preço de cada ação da empresa indicada para o preço indicado
  - **Exemplo:** `stock IBM 500.75`

4. **Listar utilizadores**
  - **Comando:** `users`
  - **Descrição:** Lista todos os utilizadores registados no sistema, indicando o username, saldo e estado (Online ou Offline)
  - **Exemplo:** `users`
5. **Pausar operações de compra/venda**
  - **Comando:** `pause <num_segundos>`
  - **Descrição:** Pausa todas as operações de compra e venda da aplicação, negando qualquer pedido que seja efetuado durante o tempo indicado.
  - **Exemplo:** `pause 30`
6. **Pausar Operações**
  - **Comando:** `pause <número-segundos>`
  - **Descrição:** Pausa todas as operações de compra e venda por um período especificado (em segundos). Informa os utilizadores caso tentem efetuar alguma compra/venda nesse período.
  - **Exemplo:** `pause 60`
7. **Encerrar a plataforma**
  - **Comando:** `close`
  - **Descrição:** Encerra o sistema, informando os clientes e boards.
  - **Exemplo:** `close`
8. **Ajuda com os comandos/sintaxe**
  - **Comando:** `help`
  - **Descrição:** Mostra a sintaxe de todos os comandos e a descrição dos mesmos
  - **Exemplo:** `help`

## Cliente

### Comandos Disponíveis

1. **Login**
  - **Comando:** `login <username> <password>`
  - **Descrição:** Autentica um utilizador no sistema.
  - **Exemplo:** `login joao 1234`
2. **Listar Empresas**
  - **Comando:** `listc`
  - **Descrição:** Lista todas as empresas registradas na Server, mostrando o nome, o número de ações disponíveis e o preço atual das ações.
  - **Exemplo:** `listc`
3. **Comprar Ações**
  - **Comando:** `buy <nome-empresa> <número-ações>`
  - **Descrição:** Permite ao utilizador comprar um número especificado de ações de uma determinada empresa.
  - **Exemplo:** `buy SpaceY 50`
4. **Vender Ações**
  - **Comando:** `sell <nome-empresa> <número-ações>`
  - **Descrição:** Permite ao utilizador vender um número de ações de uma empresa.
  - **Exemplo:** `sell IBM 20`

#### 5. Consultar Saldo

- **Comando:** `balance`
- **Descrição:** Mostra o saldo atual do utilizador e, se tiver alguma, a carteira de ações de cada empresa que obter
- **Exemplo:** `balance`

#### 6. Encerrar Sessão

- **Comando:** `exit`
- **Descrição:** Encerra a sessão do cliente e notifica o Server
- **Exemplo:** `balance`

## Board

A aplicação board não tem nenhum comando a ser executado: este apenas é lançado com um número de empresas, sendo que irá seleccionar (dentro das empresas existentes no sistema) as que obtêm um melhor valor por ação, ordenando as mesmas por ordem decrescente. Ao longo da execução, mostra as transações que serão realizadas (relativas ao “top empresas” que tem).

# Mecanismos de Sincronização

Para este projeto, usamos vários mecanismos de sincronização que aprendemos, incluindo **Mutex**, **Critical Sections**, **Semaphores**, **Waitable Timers** e **Eventos**.

**Mutex** - Usamos Mutexes para garantir que apenas um utilizador por vez possa aceder a recursos importantes, como alterar preços das ações. Isso ajuda a prevenir erros quando muitas operações acontecem ao mesmo tempo. Usados para efeitos relacionados com este exemplo que possam ocorrer em diferentes processos. Principalmente Shared Memory.

**Critical Sections** - As Critical Sections são usadas para proteger partes do código que lidam com informações sensíveis, como os saldos dos utilizadores. Elas são mais leves que os Mutexes e boas para quando poucas threads acedem ao mesmo recurso no mesmo processo.

**Semaphores** - Os Semaphores ajudam a controlar o acesso a recursos limitados. Por exemplo, garantem que não tenhamos mais clientes a interagir com o sistema do que o número máximo permitido, como uma lista de espera.

**Waitable Timers e Eventos** - Utilizamos Timers e Eventos para lidar com ações que dependem de tempo ou que precisam esperar por um evento específico, como atualizações de preços ou conclusão de vendas. O Evento pode ser usado para avisar a Board em certas circunstâncias

# Bibliografia

GitHub, Inc. "GitHub." *GitHub: Let's build from here* · GitHub, 2024, <https://github.com/>.

Accessed May 2024.

Microsoft. "Programming reference for the Win32 API - Win32 apps." *Microsoft Learn*, 2024,

<https://learn.microsoft.com/en-us/windows/win32/api/>. Accessed May 2024.

Nunes, José Luís. *Apontamentos das aulas Teóricas/Práticas*. 2024, Coimbra, Portugal.

*Inforestudante*, <https://inforestudante.ipc.pt/>. Accessed 2024.